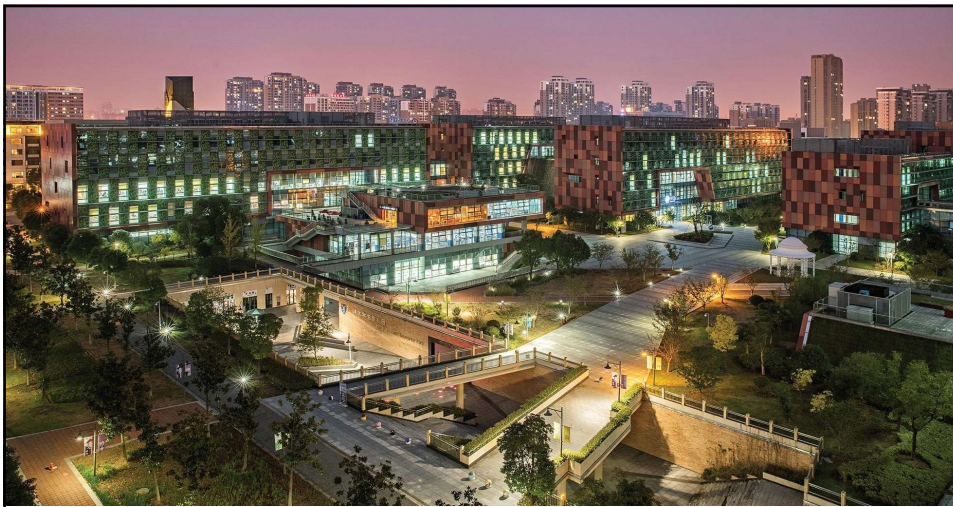




ASSOCIATIVE MEMORIES & HOPFIELD NETWORK

INT301 Bio-computation, Week 12, 2025



1

Memory in Computer System

- Standard computer memory is accessed through **assigned addresses**.
- When a user searches for a file, the CPU must convert the request to a numerical instruction and then search through the memory for the corresponding address
- A computer's memory is most commonly referred to as RAM (random access memory).

2

Associative Memory & Pattern Association

- An associative memory is a **content-addressable structure** that maps a set of input patterns to a set of output patterns.
- **That is, memory can be directly accessed by the content, rather than the physical address in the memory.**

3

Associative Memory & Pattern Association

- Associative memory is often linked to **pattern association**
 - Associating patterns which are
 - similar
 - contrary
 - in close proximity (spatial)
 - in close succession (temporal)
 - Associative recall
 - evoke associated patterns
 - recall a pattern by part of it
 - evoke/recall with incomplete/noisy patterns

4

Associative Memories

- **Two types of associative memory:** *auto-associative* and *hetero-associative*.
- Auto-association
 - retrieves a previously stored pattern that most **closely resembles** the current pattern.
- Hetero-association
 - the retrieved pattern is, in general, **different** from the input pattern not only in content but possibly also in type and format.

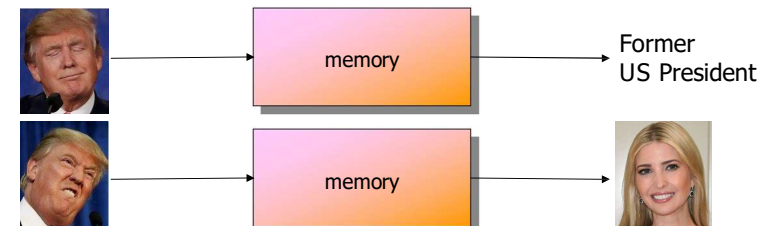
5

Associative Memories

Auto-association



Hetero-association



6

Associative Memories

Stored Patterns

$(\mathbf{x}^1, \mathbf{y}^1)$
 $(\mathbf{x}^2, \mathbf{y}^2)$
 $\vdots$
 $(\mathbf{x}^p, \mathbf{y}^p)$

$\mathbf{x}^i \in \mathbf{y}^i$

Autoassociative

$\mathbf{x}^i \in \mathbf{S} \mathbf{y}^i$

Heteroassociative

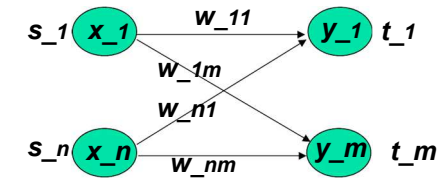
$$\mathbf{x}^i \in R^n$$

$$\mathbf{y}^i \in R^m$$

7

Simple AM

- Network structure: single layer
 - one output layer of non-linear units and one input layer
 - similar to the simple network for classification



- Goal of learning:
 - to obtain a set of weights $\mathbf{w}_{ij}$
 - from a set of training pattern pairs $\{\mathbf{s} : \mathbf{t}\}$
 - such that when $\mathbf{s}$ is applied to the input layer, $\mathbf{t}$ is computed at the output layer

8

Simple AM

- Similar to Hebbian learning for classification
- Algorithm: (bipolar or binary patterns)
 - For each training samples $\mathbf{s}:\mathbf{t}$ $w_{ij} = s_i \cdot t_j$
 - w_j increases if both input and output are ON (binary) or have the same sign (bipolar)
- If $w_j = 0$ initially, then after updates for all P training patterns

$$w_{ij} = \sum_{p=1}^P s_i(p) t_j(p) \quad W = \{w_{ij}\}$$
- Instead of obtaining $\mathbf{W}$ by iterative updates, it can be computed from the training set by calculating the outer product of $\mathbf{s}$ and $\mathbf{t}$.

9

Simple AM

- **Outer product:** Let $\mathbf{s}$ and $\mathbf{t}$ be **row** vectors.

Then for a particular training pair $\mathbf{s}:\mathbf{t}$

$$W(p) = \mathbf{s}^T(p) \otimes \mathbf{t}(p) = \begin{bmatrix} s_1 & \dots & s_n \end{bmatrix} \begin{bmatrix} t_1 & \dots & t_m \end{bmatrix} = \begin{bmatrix} s_1 t_1 & \dots & s_1 t_m \\ \vdots & \ddots & \vdots \\ s_n t_1 & \dots & s_n t_m \end{bmatrix} = \begin{bmatrix} w_{11} & \dots & w_{1m} \\ \vdots & \ddots & \vdots \\ w_{n1} & \dots & w_{nm} \end{bmatrix}$$

and $W(P) = \sum_{p=1}^P \mathbf{s}^T(p) \otimes \mathbf{t}(p)$

- It involves 3 nested loops p, i, j (order of p is irrelevant)


```

p= 1 to P          /* for every training pair */
  i = 1 to n        /* for every row in W */
    j = 1 to m      /* for every element j in row i */
       $w_{ij} := w_{ij} + s_i(p) \cdot t_j(p)$ 
    
```

10

Simple AM

- Does this method provide a good association?
 - Recall with training samples (after the weights are learned or computed)
 - Apply $s(k)$ to one layer, hope $t(k)$ appear on the other, e.g. $f(s(k)W) = t(k)$
 - May not always succeed (each weight contains some information from all samples)

$$s(k)W = s(k) \sum_{p=1} s^T(p) t(p) = \sum_{p=1} s(k) s^T(p) t(p)$$

$$= s(k) s^T(k) t(k) + \sum_{p \neq k} s(k) s^T(p) t(p)$$

$$= \underbrace{\|s(k)\|^2 t(k)}_{\text{principal term}} + \underbrace{\sum_{p \neq k} s(k) s^T(p) t(p)}_{\text{cross-talk term}}$$

11

Simple AM

- Principal term** gives the association between $s(k)$ and $t(k)$.
- Cross-talk** represents correlation between $s(k):t(k)$ and other training pairs. When cross-talk is large, $s(k)$ will recall something other than $t(k)$.
- If all $s(p)$ are orthogonal to each other, then $s(k) s^T(p) = 0$, no sample other than $s(k):t(k)$ contribute to the result.
- However, there are at most n orthogonal vectors in an n -dimensional space.
- Cross-talk increases when P increases.

12

Example 1: hetero-associative

- Binary pattern pairs $\mathbf{s}:\mathbf{t}$ with $|\mathbf{s}| = 4$ and $|\mathbf{t}| = 2$.
- Total weighted input to output units: $y_in_j = \sum_i x_i w_{ij}$
- Activation function: threshold

$$y_j = \begin{cases} 1 & \text{if } y_in_j > 0 \\ 0 & \text{if } y_in_j \leq 0 \end{cases}$$

- Weights are computed (sum of outer products of all training pairs)

$$\mathbf{W} = \sum_{p=1}^P \mathbf{s}_i^T(p) \mathbf{t}_j(p)$$

- Training samples:

	$\mathbf{s}(p)$	$\mathbf{t}(p)$
p=1	(1 0 0 0)	(1, 0)
p=2	(1 1 0 0)	(1, 0)
p=3	(0 0 0 1)	(0, 1)
p=4	(0 0 1 1)	(0, 1)

13

Example 1: hetero-associative

$$\begin{array}{lcl}
 \begin{array}{cc} (1 & 0 \\ 0 & 0 \end{array} & s^T(1) \otimes t(1) = & \begin{array}{cc} (1 & 0 \\ 0 & 0 \end{array} \\
 \begin{array}{cc} (1 & 0 \\ 0 & 0 \end{array} & s^T(2) \otimes t(2) = & \begin{array}{cc} (1 & 0 \\ 0 & 0 \end{array} \\
 \begin{array}{cc} (0 & 0 \\ 0 & 0 \end{array} & s^T(3) \otimes t(3) = & \begin{array}{cc} (0 & 0 \\ 0 & 0 \end{array} \\
 \begin{array}{cc} (0 & 0 \\ 1 & 0 \end{array} & s^T(4) \otimes t(4) = & \begin{array}{cc} (0 & 0 \\ 1 & 0 \end{array}
 \end{array}$$

Computing the weights $\mathbf{W} = \sum_{p=1}^P \mathbf{s}_i^T(p) \mathbf{t}_j(p)$

$$\mathbf{W} = \begin{pmatrix} 2 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 2 \end{pmatrix}$$

14

Example 1: hetero-associative

$\mathbf{x}=(1\ 0\ 0\ 0)$

$\mathbf{x}=(0\ 1\ 0\ 0)$ similar to S(1) and S(2)

$$\begin{pmatrix} 2 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 2 \end{pmatrix} \quad \begin{pmatrix} 2 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 2 \end{pmatrix}$$

$y_1=1, y_2=0$

$y_1=1, y_2=0$

$\mathbf{x}=(0\ 1\ 1\ 0)$

$$\begin{pmatrix} 2 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 0 & 2 \end{pmatrix}$$

$y_1=1, y_2=1$

$(1\ 0\ 0\ 0), (1\ 1\ 0\ 0)$ class (1, 0)

$(0\ 0\ 0\ 1), (0\ 0\ 1\ 1)$ class (0, 1)

$(0\ 1\ 1\ 0)$ is not sufficiently similar to any class

15

Example 2: auto-associative

- Same as hetero-associative nets, except $\mathbf{t}(\mathbf{p}) = \mathbf{s}(\mathbf{p})$.
- Used to recall a pattern by its noisy or incomplete version. (**pattern completion/pattern recovery**)
- A single pattern $\mathbf{s} = (1, 1, 1, -1)$ is stored (weights computed by outer product)

$$\mathbf{W} = \begin{pmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 \end{pmatrix}$$

training pat.	$(111-1)$	$\mathbf{W} = (4\ 4\ 4\ -4)$	$(111-1)$
noisy pat	$(-111-1)$	$\mathbf{W} = (2\ 2\ 2\ -2)$	$(111-1)$
missing info	$(001-1)$	$\mathbf{W} = (2\ 2\ 2\ -2)$	$(111-1)$
more noisy	$(-1-11-1)$	$\mathbf{W} = (0\ 0\ 0\ 0)$	not recognized

16

Example 2: auto-associative

- $\mathbf{W}$ is always a symmetric matrix
- Diagonal elements will dominate the computation when multiple patterns are stored ($= P$).
- When P is large, $\mathbf{W}$ is close to an identity matrix. This causes output = input, which may not be any stored pattern. The pattern correction power is lost.
- Replace diagonal elements by zero:

$$\begin{array}{rclcl}
 & 0 & 1 & 1 & -1 & (1 & 1 & 1 & -1)W' = (3 & 3 & 3 & -3) & (1 & 1 & 1 & -1) \\
 W' = & 1 & 0 & 1 & -1 & (-1 & 1 & 1 & -1)W' = (3 & 1 & 1 & -1) & (1 & 1 & 1 & -1) \\
 & 1 & 1 & 0 & -1 & (0 & 0 & 1 & -1)W' = (2 & 2 & 1 & -1) & (1 & 1 & 1 & -1) \\
 & -1 & -1 & -1 & 0 & (-1 & -1 & 1 & -1)W' = (1 & 1 & -1 & 1) & \text{wrong} & 17
 \end{array}$$

The Hopfield Network

- In 1982, Hopfield, a Caltech physicist, mathematically tied together many of the ideas from previous research.
- A fully connected, symmetrically weighted network where each node functions both as input and output node.
- Used for
 - Associated memories
 - Combinatorial optimization
- Major contribution of John Hopfield to NN
 - Treating a network as a dynamic system
 - Introduced the notion of energy function and attractors into NN research

18

The Hopfield Network

- Different forms: discrete & continuous
- We will focus on the **discrete** Hopfield model, because its mathematical description is more straightforward.
- In the discrete model, the output of each neuron is either 1 or -1.
- In its simplest form, the output function is the **sign function**, which yields 1 for arguments ≥ 0 and -1 otherwise.

19

Discrete Hopfield Network

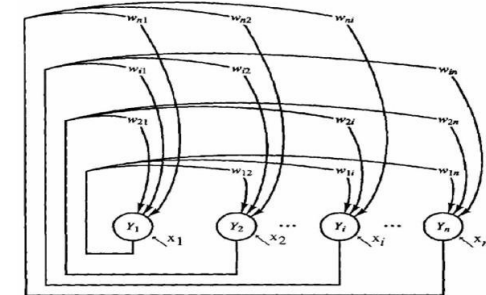
■ Architecture:

- single layer (units serve as both input and output)
- nodes are threshold units (binary or bipolar)
- weights: fully connected, symmetric, and zero diagonal

$$w_{ij} = w_{ji}$$

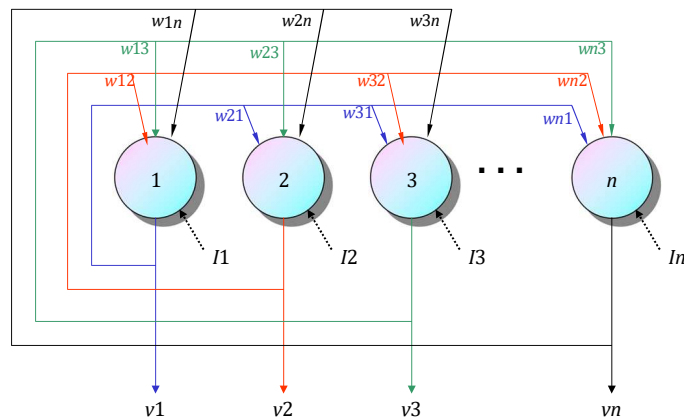
$$w_{ii} = 0$$

- x_i are external inputs, which may be transient or permanent



20

Discrete Hopfield Network



$$w_{ij} = w_{ji}$$

$$w_{ii} = 0$$

21

Discrete Hopfield Network

- Storage is performed according to the following equation:

$$w_{ij} = \frac{1}{N} \sum_{p=1}^P x_i^p x_j^p$$

- The weight matrix is symmetrical, i.e., $w_{ij} = w_{ji}$.
- The constraint condition $w_{ii} = 0$ is important for the network behavior. It can be mathematically proven that *under these conditions the network will reach a stable activation state within an infinite number of iterations.*

22



Discrete Hopfield Network

- In the discrete version of the model, each component of an input or output vector can only assume the values 1 or -1.
- The output of a neuron i at time t is then computed according to the following formula:

$$v_i(t) = \text{sgn} \left(\sum_{j=1}^N w_{ij} v_j(t-1) \right)$$

- This recursion can be performed over and over again.

23



Discrete Hopfield Network

- What does such a stable state look like?
- The network associates input patterns with themselves, which means that in each iteration, the activation pattern will be drawn towards one of those patterns.
- After converging, the network will most likely present one of the patterns that it was initialized with.
- Therefore, Hopfield networks can be used to **restore incomplete or noisy** input patterns.

24

Discrete Hopfield Network

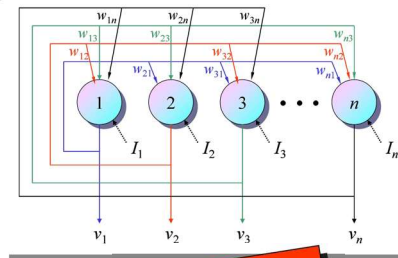
Recall

- Use an input vector to recall a stored vector
- Each time, randomly select a unit for update
- Periodically check for convergence (stable state)

Asynchronous mode update rule

$$H_i(t+1) = \sum_{j=1}^n w_{ij} v_j(t) + I_i$$

$$v_i(t+1) = \text{sgn}[H_i(t+1)] = \begin{cases} 1 & H_i(t+1) \geq 0 \\ -1 & H_i(t+1) < 0 \end{cases}$$



Stable?

25

Example:

- A 4 node network, stores 2 patterns (1 1 1 1) and (-1 -1 -1 -1)
- Weights: $w_{\ell,j} = 1$, for $\ell \neq j$, and $w_{j,j} = 0$ for all j

- Corrupted input pattern: (1 1 1 -1)

Node

selection

output

pattern

node 2: $w_{2,1}x_1 + w_{2,3}x_3 + w_{2,4}x_4 + I_2 = 1 + 1 - 1 + 1 = 2$ (1 1 1 -1)

node 4: $1 + 1 + 1 - 1 = 2$ (1 1 1 1)

No more change of state will occur, the correct pattern is recovered

- Equal distance: (1 1 -1 -1)

node 2: net = 0, no change (1 1 -1 -1)

node 3: net = 0, change state from -1 to 1 (1 1 1 -1)

node 4: net = 0, change state from -1 to 1 (1 1 1 1)

No more change of state will occur, the correct pattern is recovered

If a different node selection order is used, the stored pattern (-1 -1 -1 -1) may be recalled

26

- Missing input element: (1 0 -1 -1)

Node selection

node 2: $w_{12}x_1 + w_{32}x_3 + w_{42}x_4 + I_2 = 1 - 1 - 1 + 0 < 0$

node 1: net = -3, change state to -1

No more change of state will occur, the correct pattern is recovered

output pattern

(1 -1 -1 -1)

(-1 -1 -1 -1)

- Missing input element: (0 0 0 -1)

the correct pattern (-1 -1 -1 -1) is recovered

This is because the AM has only 2 attractors

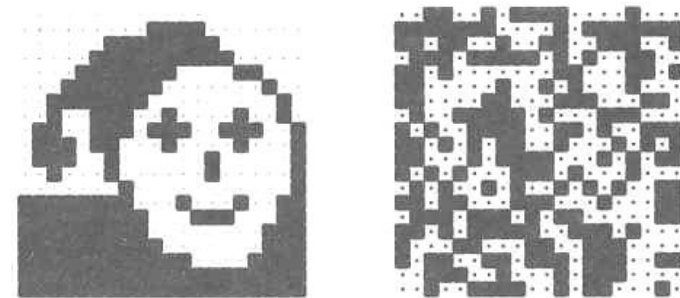
(1 1 1 1) and (-1 -1 -1 -1)

When spurious attractors exist (with more memories), pattern completion may be incorrect

27

Example

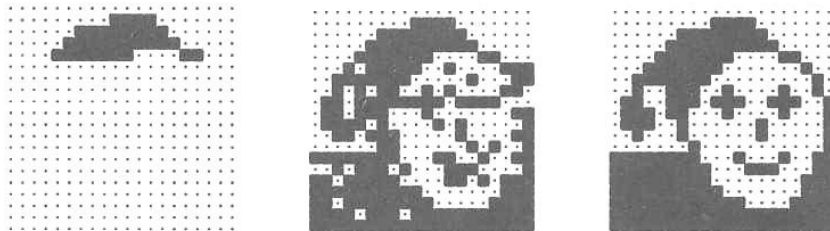
- Image reconstruction (Ritter, Schulten, Martinetz 1990)
- A 20 × 20 discrete Hopfield network was trained with 20 input patterns, including the one shown in the left figure and 19 random patterns as the one on the right.



28

Example

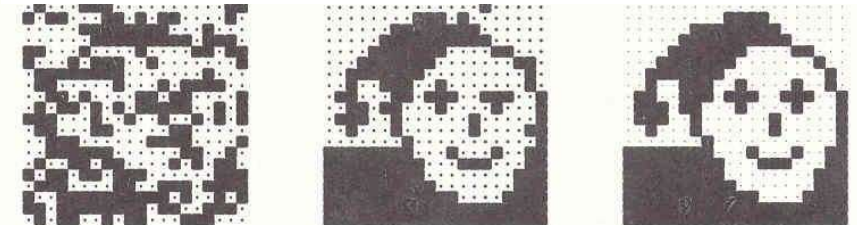
- After providing only one fourth of the “face” image as initial input, the network is able to perfectly reconstruct that image within only two iterations.



29

Example

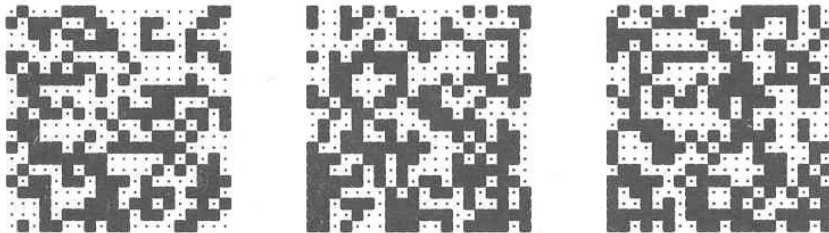
- Adding noise by changing each pixel with a probability $p = 0.3$ does not impair the network's performance.
- After two steps the image is perfectly reconstructed.



30

Example

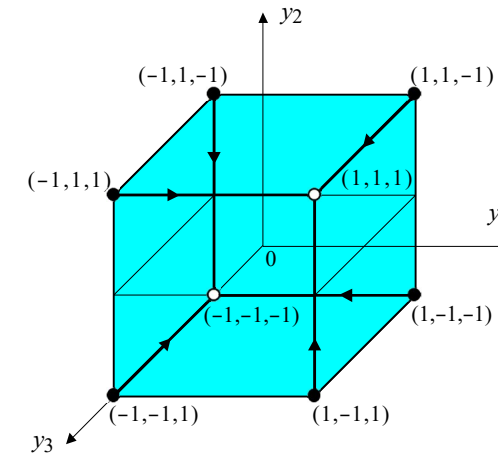
- However, for noise created by $p = 0.4$, the network is unable to restore the original image.
- Instead, it converges against one of the 19 random patterns.



31

Discrete Hopfield Network

Possible 8 states for the three-neuron Hopfield network



32

Discrete Hopfield Network

- The stable state is determined by the weight matrix $\mathbf{W}$, the current input vector $\mathbf{X}$, and the threshold matrix $\mathbf{q}$. If the input vector is partially incorrect or incomplete, the initial state will converge into the stable state after a few iterations.
- Suppose, for instance, that the network is required to memorize two opposite states, (1, 1, 1) and (-1, -1, -1). Thus,

$$\mathbf{Y}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \quad \mathbf{Y}_2 = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix} \quad \text{or} \quad \mathbf{Y}_1^T = [1 \ 1 \ 1] \quad \mathbf{Y}_2^T = [-1 \ -1 \ -1]$$

33

Discrete Hopfield Network

- To build the weight matrix

$$\mathbf{W} = \sum_{k=1}^p \mathbf{x}^k (\mathbf{x}^k)^T - p\mathbf{I} \quad w_{ij} = \begin{cases} \sum_{k=1}^p x_i^k x_j^k & i \neq j \\ 0 & i = j \end{cases}$$

- Determine the weight matrix as follows:

$$\mathbf{W} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} + \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix} \begin{bmatrix} -1 & -1 & -1 \end{bmatrix} - 2 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 2 & 2 \\ 2 & 0 & 2 \\ 2 & 2 & 0 \end{bmatrix}$$

- Next, the network is tested by the sequence of input vectors, $\mathbf{X}_1$ and $\mathbf{X}_2$, which are equal to the output (or target) vectors $\mathbf{Y}_1$ and $\mathbf{Y}_2$, respectively.

34

Discrete Hopfield Network

- Activate the Hopfield network by applying input vector X and calculate the actual output vector Y

$$Y_1 = \text{sign} \begin{pmatrix} 0 & 2 & 2 & 1 & 0 & \ddot{u} & 1 \\ 2 & 0 & 2 & 1 & - & 0 & \hat{y} = 1 \\ 2 & 2 & 0 & 1 & 0 & j & 1 \end{pmatrix}$$

$$Y_2 = \text{sign} \begin{pmatrix} 0 & 2 & 2 & -1 & 0 & \ddot{u} & -1 \\ 2 & 0 & 2 & -1 & - & 0 & \hat{y} = -1 \\ 2 & 2 & 0 & -1 & 0 & j & -1 \end{pmatrix}$$

- Compare the result with the initial input vector X

35

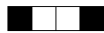
Discrete Hopfield Network

- The remaining six states are all unstable. However, stable states (also called **fundamental memories**) are capable of attracting states that are close to them.
- The fundamental memory (1, 1, 1) attracts unstable states (-1, 1, 1), (1, -1, 1) and (1, 1, -1). Each of these unstable states represents a single error, compared to the fundamental memory (1, 1, 1).
- The fundamental memory (-1, -1, -1) attracts unstable states (-1, -1, 1), (-1, 1, -1) and (1, -1, -1).
- Thus, the Hopfield network can act as an **error correction network**.

36

Example 1: Weights Matrix

$$\mathbf{x}^1 = (1, -1, -1, 1)^T$$



$$\mathbf{x}^2 = (-1, 1, -1, 1)^T$$



$$\mathbf{x}^1(\mathbf{x}^1)^T = \begin{pmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}$$

$$\mathbf{x}^2(\mathbf{x}^2)^T = \begin{pmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{pmatrix}$$

$$\mathbf{x}^1(\mathbf{x}^1)^T + \mathbf{x}^2(\mathbf{x}^2)^T = \begin{pmatrix} 2 & -2 & 0 & 0 \\ -2 & 2 & 0 & 0 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & -2 & 2 \end{pmatrix}$$

$$\mathbf{W} = \sum_{k=1}^p \mathbf{x}^k (\mathbf{x}^k)^T - p\mathbf{I}$$

$$\mathbf{W} = \begin{pmatrix} 0 & -2 & 0 & 0 \\ -2 & 0 & 0 & 0 \\ 0 & 0 & 0 & -2 \\ 0 & 0 & -2 & 0 \end{pmatrix}$$

37

Example 2: Spurious State

- Use a 4-node Hopfield network to store 3 patterns:

(1 1 -1 -1), (1 1 1 1) and (-1 -1 1 1)

weights:
$$\begin{pmatrix} 0 & 1 & -1/3 & -1/3 \\ 1 & 0 & -1/3 & -1/3 \\ -1/3 & -1/3 & 0 & 1 \\ -1/3 & -1/3 & 1 & 0 \end{pmatrix}$$

- Corrupted input pattern: (-1 -1 -1 -1)

- if node 4 is randomly selected:
 $(-1/3 \ -1/3 \ 1 \ 0) (-1 \ -1 \ -1 \ -1)^T + (-1) = 1/3 + 1/3 - 1 - 0 - 1 = -4/3$
- no change of state for node 4
- same for all other nodes, net stabilized at (-1 -1 -1 -1)
- a spurious state is recalled

38

Example 2: Spurious State

- For input pattern $(-1 \ -1 \ -1 \ 0)$
 - if node 4 is selected first
 $(-1/3 \ -1/3 \ 1 \ 0) (-1 \ -1 \ -1 \ 0)^T + (0) = 1/3 + 1/3 - 1 - 0 - 0 = -1/3$
 - node 4 changes state to -1: $(-1 \ -1 \ -1 \ -1)$
 - network stabilizes at $(-1 \ -1 \ -1 \ -1)$
 - however, if the node selection sequence is $1 > 2 > 3 > 4$, the net stabilizes at state $(-1 \ -1 \ 1 \ 1)$: a correct pattern

$$W = \begin{pmatrix} 0 & 1 & -1/3 & -1/3 \\ 1 & 0 & -1/3 & -1/3 \\ -1/3 & -1/3 & 0 & 1 \\ -1/3 & -1/3 & 1 & 0 \end{pmatrix}$$

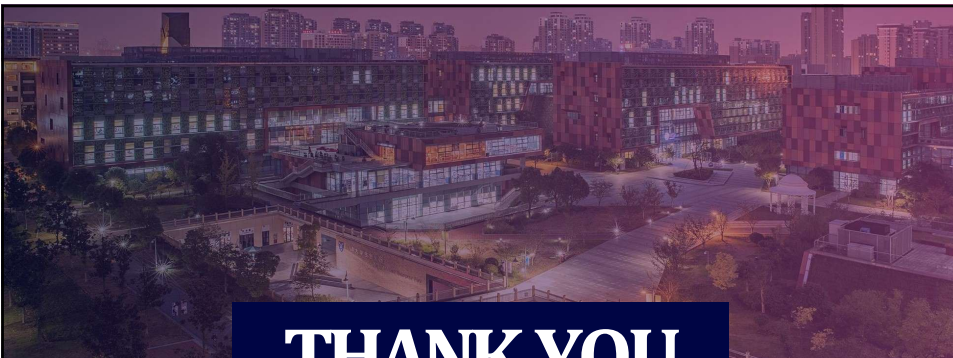
-1	-1	-1	0
-1	-1	-1	0
-1	-1	-1	0
-1	-1	1	0
-1	-1	1	1
-1	-1	1	1
-1	-1	1	1
-1	-1	1	1

39

Limitations of Hopfield Network

- The number of patterns that can be stored and accurately recalled is severely limited
 - net may converge to a novel spurious pattern
- Exemplar pattern will be unstable if it shares many bits in common with another exemplar pattern

40



THANK YOU

VISIT US

WWW.XJTLU.EDU.CN

FOLLOW US

@XJTLU

 Xi'an Jiaotong-Liverpool University
西交利物浦大学

41